

ALC Item Sheet Operating Manual

Quick Navigation

- Purpose and operating model
- Daily behavior and script prompts
- Core workflows (asset entry, invoicing, bank payable, month-end)
- Baseline initialization and compliance artifacts
- Official file locations and short operator commands
- Reports, duties by role, and decision flows
- Schedule and ledger rules

Purpose

This manual defines the daily, weekly, month-end, and exception-handling workflow for Acento Leasing Company (ALC) asset invoicing and lease tracking.

The system is designed to be simple:

- One script
- One asset table
- One rate table
- One projected amortization schedule
- One posted invoice ledger
- One daily report
- One monthly invoice run
- One bank-payable run

Operating Model

The script should support two modes:

1. Guided mode

- The script asks the operator a short sequence of questions.
- It only asks questions when action is needed.
- It then routes the operator to the next required task.

2. Report-only mode

- If nothing changed and no billing action is due, the script skips questions.

- It generates the daily report, inventory snapshot, and maturity summary.
- It exits cleanly with a message such as: "No operational actions required today."

Official File Locations

The script now defaults to one official set of files inside the ALC folder:

- assets input: ALC/assets.csv
- rates input: ALC/rates.csv
- posted ledger: ALC/posted_invoices.csv
- closed periods: ALC/closed_periods.csv
- baseline config: ALC/baseline_config.json
- manifests folder: ALC/run_manifests
- backups folder: ALC/backups
- one-pager workbook: [ALC/ALC - Asset Calculation Unit.xlsx](ALC/ALC%20-%20Asset%20Calculation%20Unit.xlsx)

CLI overrides are still available for admin/testing use, but operators should use the official default paths.

Official Operator Command Set

Short wrapper scripts are available in ALC/scripts:

- baseline reset: ALC/scripts/op_init_baseline.sh
- daily run: ALC/scripts/op_daily.sh
- schedule review: ALC/scripts/op_schedule.sh
- month-end invoice: ALC/scripts/op_invoice.sh
- bank payable: ALC/scripts/op_bank_payable.sh
- one-pager workbook: ALC/scripts/op_one_pager.sh

Examples:

- baseline: `ALC/scripts/op_init_baseline.sh german 2026-07-01 "Production start"``
- daily: `ALC/scripts/op\_daily.sh german 2026-07-02`
- invoice: `ALC/scripts/op\_invoice.sh german 2026-07`
- bank payable: `ALC/scripts/op\_bank\_payable.sh german 2026-07`
- one-pager: `ALC/scripts/op\_one\_pager.sh german`

Daily Behavior

Every day the operator runs the script once.

The script checks, in this order:

1. New assets added today?
2. Any asset status changes?
3. Any new bank rate effective today?
4. Any invoices due today?
5. Any bank payment due today?
6. Any exceptions or mismatches?

If the answer is "no" to all of the above, the script produces only the report package.

If the answer is "yes" to any item, the script prompts for the missing data and then continues.

What the Script Should Ask

The script should only ask for facts the system cannot infer.

Typical questions:

- Did any new asset enter service today?
- Was any asset disposed, transferred, or deactivated?
- Did the bank publish a new annual rate?
- Are we closing the month today?
- Do you want to generate invoices now?
- Do you want to generate the bank payment summary now?
- Do you want to export CSV, PDF, or both?

The script should not ask repetitive questions if no action is needed.

Core Workflows

1. Asset Entry Workflow

When a new asset is purchased or allocated:

- Create the asset record
- Assign property and allocation
- Set start date and lease term
- Set asset value, tax, admin expense, risk recovery, and salvage value
- Set salvage periods when a final residual month is reserved
- Assign bank APR and NIM assumption

- Assign GL account
- Mark asset as active

The system then calculates:

- lease base
- projected monthly installment table
- expected maturity date
- opening balance
- projected amortization schedule

The projected schedule uses the entry-time bank APR plus NIM across the full remaining term until a month-specific bank rate is loaded.

The lease math is:

- lease base = asset value + tax + admin expense + risk cost recovery - salvage value
- lease rate = effective monthly rate derived from annual APR + NIM
- payment months = term months - salvage periods

2. Daily Operations Workflow

Each morning the operator runs the daily script.

The script outputs:

- active asset count
- assets starting soon
- assets maturing soon
- invoices due today
- bank payments due today
- assets with missing fields
- balance reconciliation totals

For any month already invoiced, the script reads the posted invoice ledger first and treats those rows as locked historical actuals.

If no action is needed, the operator simply reviews the report and closes the task.

3. Monthly Invoicing Workflow

Use one monthly billing cycle for all active assets.

Recommended policy:

- Close billing on the last day of the month

- Invoice all active assets as of the close date
- Include line-item detail by asset or allocation
- Group by property for customer invoicing

At invoicing time the script should:

- read the projected schedule for the target month
- replace the target-month rate with the current bank rate if a month-specific rate exists
- generate invoice lines from that month's schedule rows
- write the billed rows to the posted invoice ledger
- keep those posted rows fixed in future runs

This is easier than anniversary billing because it keeps accounting, collections, and bank reconciliation aligned.

4. Bank Payment Workflow

The system should also create a monthly bank-payable summary.

That summary shows:

- principal due
- interest due
- total debt service due
- outstanding balance by asset
- total portfolio balance

The operator uses this to pay the bank and cross-check the debt ledger.

5. Month-End Close Workflow

At month-end the operator should:

- confirm all asset changes are entered
- confirm the bank rate table is current
- review the projected schedule for reasonableness
- generate invoices
- update the posted invoice ledger
- generate bank payable report
- export reports to accounting
- save a locked month snapshot

After close, changes should be handled by adjustment entries, not by editing closed rows.

6. Baseline Initialization Workflow

Use this once before production start (example: go-live on 2026-07-01).

Recommended baseline steps:

- Back up current working files.
- Initialize baseline config with go-live date.
- Reset posted invoice ledger for clean production history.
- Reset closed periods file.
- Store the baseline run manifest.

Suggested command pattern:

- run ``init-baseline`` with ``--go-live YYYY-MM-DD``
- include ``--reset-posted-ledger`` and ``--reset-closed-periods`` for clean start
- include ``--operator`` for traceability

After baseline:

- Invoicing before go-live is blocked unless explicitly authorized.
- Closed months are blocked unless ``allow-closed-adjustment`` is explicitly used.

Reports

Daily Reports

1. Portfolio summary
2. Active asset inventory
3. Maturity schedule
4. Invoice due list
5. Bank payable due list
6. Exception list
7. Reconciliation summary

Monthly Reports

1. Invoice register
2. Property-level billing summary
3. Asset-level amortization report
4. Projected amortization schedule
5. Posted invoice ledger
6. Bank debt summary

7. Closed-month snapshot
8. Variance report

Compliance Artifacts

The script now maintains operational traceability files:

- ``baseline_config.json``: go-live date and baseline metadata
- ``posted_invoices.csv``: posted invoice ledger (historical actuals)
- ``closed_periods.csv``: list of closed months (``YYYY-MM``)
- ``run_manifests/*.json``: one manifest per command run
- ``backups//...``: file snapshots created before write operations

Each manifest should include:

- run ID
- operator
- command and timestamp
- key totals and row counts
- input file hashes (before and after when applicable)
- backup file references

Exception Reports

1. Asset missing start date
2. Asset missing GL account
3. Asset with zero or negative balance unexpectedly
4. Rate change with no effective date
5. Invoice mismatch
6. Bank balance mismatch

Duties by Role

Operator / Agent

- Run the script daily
- Review exception list
- Enter new assets
- Confirm rate changes
- Review projected schedule for new assets
- Launch invoicing on month-end

- Launch bank payable run
- Save or export reports

Accounting / Finance

- Review invoices
- Post receivables
- Confirm bank payment
- Reconcile ledger vs bank debt
- Approve adjustment entries

Management

- Review summary dashboards
- Watch active inventory
- Monitor maturity exposure
- Track revenue and debt trends

Recommended System Behavior

The script should behave like a guided checklist:

- If a task is needed, prompt for it
- If a task is not needed, skip it
- If no action is needed at all, generate reports and exit

That means the operator always has a clear next step.

Suggested Daily Decision Flow

1. Load master data.
2. Check for new assets.
3. Check for rate changes.
4. Check for invoices due.
5. Check for bank payment due.
6. If action exists, prompt the operator.
7. If no action exists, generate the daily report package.
8. Exit.

Suggested Month-End Decision Flow

1. Are all assets entered?
2. Are all rates updated?
3. Has the projected schedule been reviewed?
4. Generate invoices.
5. Post the invoice rows to the invoice ledger.
6. Generate bank payable report.
7. Export closed-month package.
8. Lock the month.

Schedule and Ledger Rules

- The projected amortization schedule may change for the current month and future months when a new bank rate is loaded.
- A posted invoice row is a historical actual and must not be recalculated by later runs.
- Future months should always roll forward from the ending balance of the latest posted month.
- If a closed month needs correction, post an adjustment entry instead of editing the posted ledger row.

Operating Principle

The company should be run from the report, not from memory.

If the report is clean, no action is needed.

If the report shows an exception, the script should surface it immediately and tell the operator what to do next.